

Unit 10: Testing Levels

CONTENTS

Objective

Introduction

10.1 Unit Testing

10.2 Integration Testing

10.3 System Testing

10.4 User Acceptance Testing

10.5 Regression Testing

Summary

Keywords

Self Assessment

Answer for Self Assessment

Review Questions

Further Reading

Objective

- after studying this unit, you will be able to:
- explain unit testing and integration testing
- discuss system testing and acceptance testing
- regression testing

Introduction

Software testing is an investigation that is carried out to offer information to stakeholders regarding the quality of the product or service being tested. Software testing can also give a corporation with an objective, unbiased picture of the software, allowing them to grasp and comprehend the risks associated with software implementation. The process of executing a programme or application with the purpose of detecting software bugs is one example of a test technique (errors or other defects). Testing is the process of assessing a system or its component(s) with the goal of determining whether or not it meets the set requirements. Testing is executing a system in order to identify any gaps, errors or missing requirements in contrary to the actual desire or requirements.

10.1 Unit Testing

Unit testing is the basic level of testing. Unit testing is concerned with the verification of individual software components or modules. Important control pathways are identified and evaluated using the design to discover faults within the module border. It entails isolating a module from the rest of the software, creating test cases, and comparing the actual results to the expected results as defined by the specifications and design. One of its goals is to locate and correct as many software faults as possible.

A unit test is a way of testing a unit - the smallest piece of code that can be logically isolated in a system. In most programming languages, that is a function, a subroutine, a method or property. The isolated part of the definition is important. In his book "Working Effectively with Legacy Code", author Michael Feathers states that such tests are not unit tests when they rely on external systems: "If it talks to the database, it talks across the network, it touches the file system, it requires system configuration, or it can't be run at the same time as any other test."

Unit testing is the process of testing each unit or component of a software application separately. Unit testing is carried out by developers throughout the development (coding) phase of an application. Unit tests are used to isolate a part of code and ensure that it is correct. A singular function, method, process, module, or object might be considered a unit. Unit testing is a type of White Box testing.

Why Unit Testing

- Unit tests help to fix bugs early in the development cycle and save costs.
- It helps the developers to understand the testing code base and enables them to make changes quickly
- Good unit tests serve as project documentation
- Unit tests help with code re-use. Migrate both your code and your tests to your new project.

Work flow of unit testing

- Creating test cases
- Reviewing test cases
- Base lining test cases
- Executing test cases

Types of Unit testing

Manual: The manual approach has a step by step instructional procedure that helps testers perform this task efficiently.

Automated: The automated method is the most preferred as it is faster and more accurate. A developer writes a section of code in the application just to test the function. Isolate the function to test it more rigorously. Isolating the code helps in revealing unnecessary dependencies between the code being tested and other units or data spaces in the product. Developer uses a Unit Test Framework to develop automated test cases and codes criteria into the test to verify the correctness of the code.

Unit Testing Techniques

Black box testing: that involves testing of user interface along with input and output

White box testing: that involves testing the functional behavior of the software application

Grey box testing: that is used to execute test suites, test methods, test cases and performing risk analysis.

Unit testing best practices

- Ensure Unit Tests are independent of each other
- Always stick to one code unit at once
- Make use of AAA for readability- (Arrange, Act, Assert)
- Fix the bugs before moving for Integration Testing
- Ensure to have proper variable naming
- Always separate test code and production code

Unit Testing Tools

Junit: Junit is a free to use testing tool used for Java programming language. It provides assertions to identify test method. This tool test data first and then inserted in the piece of code.

NUnit: NUnit is widely used unit-testing framework use for all .net languages. It is an open source tool which allows writing scripts manually. It supports data-driven tests which can run in parallel.

JMockit: JMockit is open source Unit testing tool. It is a code coverage tool with line and path metrics. It allows mocking API with recording and verification syntax. This tool offers Line coverage, Path Coverage, and Data Coverage.

EMMA: EMMA is an open-source toolkit for analyzing and reporting code written in Java language. Emma support coverage types like method, line, basic block. It is Java-based so it is without external library dependencies and can access the source code.

PHPUnit: PHPUnit is a unit testing tool for PHP programmer. It takes small portions of code which is called units and test each of them separately. The tool also allows developers to use pre-define assertion methods to assert that a system behave in a certain manner.

The various errors that must be checked for while testing are as under:

1. Error description is unintelligible.
2. Error noted does not correspond to error encountered.
3. Error condition causes system intervention prior to error handling.
4. Exception-condition processing is incorrect.
5. Error description does not provide enough information to assist in the location of the cause of the error.

Advantages of Unit testing

- Isolates a section of code and validates its correctness.
- Helps in identifying and fixing the bugs at the early stage of the SDLC process.
- Assures to reduce the cost as bugs are resolved at the earliest.
- Helps the developers to improve the design by allowing refactoring of the code.
- Assures in simplifying the debugging process.
- With the proper unit testing practice, components that are integrated after the build can assure in achieving a quality product.

Unit Testing Disadvantages

Unit testing can't be expected to catch every error in a program. It is not possible to evaluate all execution paths even in the most trivial programs.

Unit testing by its very nature focuses on a unit of code. Hence it can't catch integration errors or broad system level errors.

10.2 Integration Testing

Individual modules / components are merged and tested as a group during integration testing. This level of testing is designed to reveal flaws in the interaction of integrated units. Integration testing is concerned with ensuring that data is communicated between various units. It's also known as 'I & T' (Integration and Testing), 'String Testing,' and 'Thread Testing,' among other terms. Unit testing uses modules for testing, and integration testing combines and tests these parts.

Integration testing is a systematic technique for constructing the program structure while at the same time conducting tests to uncover errors associated with interfacing. The objective is to take unit tested components and build a program structure that has been dictated by design. There is often a tendency to attempt non-incremental integration; that is, to construct the program using a "big bang" approaches. All components are combined in advance. The entire program is tested as a whole. A set of errors is encountered. Correction is difficult because isolation of causes is complicated by the vast expanse of the entire program. Once these errors are corrected, new ones appear and the process continues in a seemingly endless loop.

The big bang technique is the polar opposite of incremental integration. The programme is built and tested in small chunks, making it easier to isolate and rectify faults; interfaces are more likely to be thoroughly verified; and a systematic testing approach can be used.

Top-down Integration

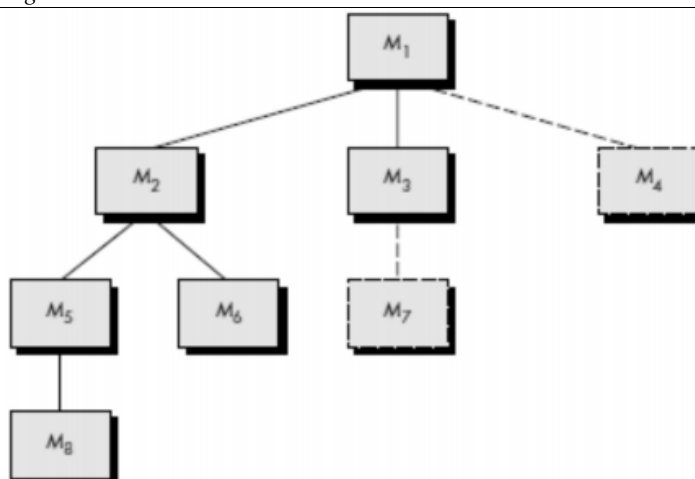
Top-down integration testing is an incremental approach to construction of program structure. Modules are integrated by moving downward through the control hierarchy, beginning with the main control module. Modules subordinate to the main control module are incorporated into the structure in either a depth-first or breadth-first manner.

Referring to Figure, depth-first integration would integrate all components on a major control path of the structure. Selection of a major path is somewhat arbitrary and depends on application-specific characteristics.

Example: Selecting the left-hand path, components M1, M2, M5 would be integrated first.

Next, M8 or if needed for proper functioning of M2) M6 would be integrated. Then, the central and right-hand control paths are built. Breadth-first integration incorporates all components directly subordinate at each level, moving across the structure horizontally. From the figure, components M2, M3, and M4 (a replacement for stub S4) would be integrated first. The next control level, M5, M6, and so on, follows.

Top down Integration



The integration process is performed in a series of five steps:

1. The main control module is used as a test driver and stubs are substituted for all components directly subordinate to the main control module.
2. Depending on the integration approach selected (i.e., depth or breadth first), subordinate stubs are replaced one at a time with actual components.
3. Tests are conducted as each component is integrated.
4. On completion of each set of tests, another stub is replaced with the real component.
5. Regression testing may be conducted to ensure that new errors have not been introduced.

The process continues from step 2 until the entire program structure is built.

Advantages - Top-Down Integration Testing

- Separately debugged module.
- Few or no drivers needed.
- Fault Localization is easier.
- Possibility to obtain an early prototype.
- Critical Modules are tested on priority; major design flaws could be found and fixed first

Bottom-Up Integration

Bottom-up integration testing, as its name implies, begins construction and testing with atomic modules i.e., components at the lowest levels in the program structure.

Because components are integrated from the bottom up, processing required for components subordinate to a given level is always available and the need for stubs is eliminated.

A bottom-up integration strategy may be implemented with the following steps:

1. Low-level components are combined into clusters (also referred as builds) that perform a specific software sub function.
2. A driver is written to coordinate test case input and output.
3. The cluster is tested.
4. Drivers are removed and clusters are combined moving upward in the program structure

Advantages - Bottom-Up Integration Testing

- In bottom-up testing, no stubs are required.
- A principle advantage of this integration testing is that several disjoint subsystems can be tested simultaneously.
- Fault localization is easier.

Big-Bang Integration Testing: In Big-Bang Integration Testing all the modules are combining and verifying the functionality after the completion of individual module testing. This approach is practicable only for very small systems. Debugging errors reported during big bang integration testing are very expensive to fix.

Mixed/ sandwiched Integration Testing: A mixed integration testing is also called sandwiched integration testing. A mixed integration testing follows a combination of top down and bottom-up testing approaches. This sandwich or mixed approach overcomes this shortcoming of the top-down and bottom-up approaches.

Incremental Testing: In the Incremental Testing approach, testing is done by integrating two or more modules that are logically related to each other and then tested for proper functioning of the application. Then the other related modules are integrated incrementally and the process continues until all the logically related modules are integrated and tested successfully.

Stubs and Drivers: Stubs and Drivers are the dummy programs in Integration testing used to facilitate the software testing activity. These programs act as a substitutes for the missing models in the testing. They do not implement the entire programming logic of the software module but they simulate data communication with the calling module while testing.

Stub: Is called by the Module under Test.

Driver: Calls the Module to be tested.

Best Practices for Integration Testing

First, determine the Integration Test Strategy that could be adopted and later prepare the test cases and test data accordingly.

Study the Architecture design of the Application and identify the Critical Modules. These need to be tested on priority.

Obtain the interface designs from the Architectural team and create test cases to verify all of the interfaces in detail. Interface to database/external hardware/software application must be tested in detail.

After the test cases, it's the test data which plays the critical role.

Always have the mock data prepared, prior to executing. Do not select test data while executing the test cases.

10.3 System Testing

There are two types of system testing: integration testing and acceptance testing. The bottom-up approach, top-down strategy, and sandwich strategy are all methods for merging software components into a working product. To ensure that modules are available for incorporation into the expanding software product when needed, careful planning and scheduling are required.

The integration strategy dictates the order in which modules must be available, and thus exerts a strong influence on the order in which modules are written, debugged, and unit tested.

Acceptance testing involves planning and execution of functional tests, performance tests, and stress tests to verify that the implemented system satisfies its requirements. Acceptance tests are typically performed by the quality assurance and/or customer organizations. Depending on local circumstances, the development group may or may not be involved in acceptance testing.

System testing is done to ensure that the software is complete and integrated. Its purpose is to assess the system's end-to-end specs. System testing looks for flaws in both the individual parts and the entire system. The observed behavior of a component or a system when it is tested is the result of system testing.

System testing is performed on the entire system, either in accordance with system requirement requirements, functional requirement specifications, or both. System testing is included in the black box testing area of software testing since it encompasses testing of the software's external functionality.

System testing is testing conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements. System testing falls within the scope of Black box testing, and as such, should require no knowledge of the inner design of the code. System testing is actually done to the entire system against the Functional Requirement Specifications (FRS) and/or the System Requirement Specification (SRS). Moreover, the System testing is an investigatory testing phase, where the focus is to have almost a destructive attitude and test not only the design, but also the behavior and even the believed expectations of the customer. It is also intended to test up to and beyond the bounds defined in the software/hardware requirements specifications. Remaining All Testing Models comes under System Testing.

Steps for System Testing

Test Environment Setup: Create testing environment for the better quality testing.

Create Test Case: Generate test case for the testing process.

Create Test Data: Generate the data that is to be tested.

Execute Test Case: After the generation of the test case and the test data, test cases are executed.

Defect Reporting: Defects in the system are detected.

Regression Testing: It is carried out to test the side effects of the testing process.

Log Defects: Defects are fixed in this step.

Retest: If the test is not successful then again test is performed.

Types of System Testing

Performance Testing: Performance Testing is a type of software testing that is carried out to test the speed, scalability, stability and reliability of the software product or application.

Load Testing: Load Testing is a type of software testing which is carried out to determine the behavior of a system or software product under extreme load.

Stress Testing: Stress Testing is a type of software testing performed to check the robustness of the system under the varying loads.

Usability Testing: mainly focuses on the user's ease to use the application, flexibility in handling controls and ability of the system to meet its objectives.

Scalability Testing: Scalability Testing is a type of software testing which is carried out to check the performance of a software application or system in terms of its capability to scale up or scale down the number of user request load.

Regression Testing: involves testing done to make sure none of the changes made over the course of the development process have caused new bugs. It also makes sure no old bugs appear from the addition of new software modules over time.

Recovery testing: is done to demonstrate a software solution is reliable, trustworthy and can successfully recoup from possible crashes.

Migration testing: is done to ensure that the software can be moved from older system infrastructures to current system infrastructures without any issues.

Functional Testing: Functional testing of a system is performed to find if there's any missing function in the system. Tester makes a list of vital functions that should be in the system and can be added during functional testing and should improve quality of the system.

10.4 User Acceptance Testing

User Acceptance Testing performed by the end user or the client to verify/accept the software system before moving the software application to the production environment. The major aim of this test is to evaluate the compliance of the system with the business requirements and assess whether it is acceptable for delivery or not.

After System Testing and before making the system accessible for actual use, Acceptance Testing is the final phase of software testing. It's done in a way that's similar to Black Box testing, with the number of users required to test the system's acceptability level..

This is arguably the most importance type of testing as it is conducted by the Quality Assurance Team who will gauge whether the application meets the intended specifications and satisfies the client's requirements. The QA team will have a set of pre written scenarios and Test Cases that will be used to test the application.

More thoughts regarding the application will be presented, and more testing will be conducted to assess its accuracy and the reasons for the project's inception. Acceptance tests are designed to identify not only basic spelling mistakes, cosmetic faults, and interface gaps, but also any bugs in the application that could cause system crashes or serious issues.

By performing acceptance tests on an application the testing team will deduce how the application will perform in production. There are also legal and contractual requirements for acceptance of the system.

Steps for User Acceptance Testing

Analysis of Business Requirements
 Creation of UAT test plan
 Identify Test Scenarios
 Create UAT Test Cases
 Preparation of Test Data (Production like Data)
 Run the Test cases
 Record the Results
 Confirm business objectives

Types of Acceptance Testing

User Acceptance Testing (UAT)
 Business Acceptance Testing (BAT)
 Contract Acceptance Testing (CAT)
 Regulations Acceptance Testing (RAT)
 Operational Acceptance Testing (OAT)
 Alpha Testing
 Beta Testing

Business Acceptance Testing: BAT is used to determine whether the product meets the business goals and purposes or not. BAT mainly focuses on business profits which are quite challenging due to the changing market conditions and new technologies so that the current implementation may have to be changed which result in extra budgets.

Contract Acceptance Testing: CAT is a contract which specifies that once the product goes live, within a predetermined period, the acceptance test must be performed and it should pass all the acceptance use cases.

Regulations Acceptance Testing: RAT is used to determine whether the product violates the rules and regulations that are defined by the government of the country where it is being released.

Operational Acceptance Testing: OAT is used to determine the operational readiness of the product and is a non-functional testing. It mainly includes testing of recovery, compatibility, maintainability, reliability etc.

Alpha testing: Alpha testing is used to determine the product in the development testing environment by a specialized tester's team usually called alpha testers.

Beta testing: Beta testing is used to assess the product by exposing it to the real end-users, usually called beta testers in their environment. Feedback is collected from the users and the defects are fixed. Also, this helps in enhancing the product to give a rich user experience.

User Acceptance testing tools

Fitness tool: It is a java tool used as a testing engine. It to create tests and record results in a table. Users of the tool enter the formatted input and tests are created automatically. The tests are then executed and the output is returned back to the user.

Watir: It is toolkit used to automate browser-based tests during User acceptance testing. Ruby is the programming language used for inter-process communication between ruby and Internet Explorer.

10.5 Regression Testing

Regression Testing is re-execute or re-running functional and non-functional tests to ensure that previously developed and tested software still performs after a change.

Regression Testing is done to make sure that new code changes should not have side effects on the existing functionalities. It ensures that the old code still works once the latest code changes are done.

Regression testing is re-running a subset of previously run tests to confirm that modifications have not resulted in unwanted side effects. Manual regression testing, re-executing a portion of all test cases, or employing automated capture or playback technologies are all options. Software engineers can use capture or playback tools to record test cases and results for later replay and comparison.

The regression test suite (the subset of tests to be executed) contains three different classes of test cases:

- A representative sample of tests that will exercise all software functions.
- Additional tests that focus on software functions that are likely to be affected by the change.
- Tests that focus on the software components that have been changed.

The number of regression tests can quickly rise as integration testing progresses. As a result, only those tests should be included in the regression test suite that address one or more classes of defects in each of the key programme functions. Once a change has occurred, it is impractical and inefficient to re-run every test for every software function.

After modifying software, either for a change in functionality or to fix defects, a regression test re-runs previously passing tests on the modified software to ensure that the modifications haven't unintentionally caused a regression of previous functionality. Regression testing can be performed at any or all of the above test levels. These regression tests are often automated.

In integration testing also, each time a module is added, the software changes. New data flow paths are established, new I/O may occur, and new control logic is invoked. Hence, there is the need of regression testing.

Any sort of software testing that aims to find regression bugs is known as regression testing. When software functionality that previously worked as expected stops working or no longer works in the same way that it was anticipated, regression bugs develop. Regression bugs are typically unintentional consequences of programme changes.

Common methods of regression testing include re-running previously run tests and checking whether previously fixed faults have reemerged. Experience has shown that as software is developed, this kind of reemergence of faults is quite common. Sometimes it occurs because a fix gets lost through poor revision control practices (or simple human error in revision control), but just as often a fix for a problem will be "fragile" - i.e. if some other change is made to the program, the fix no longer works. Finally, it has often been the case that when some feature is redesigned, the same mistakes will be made in the redesign that were made in the original implementation of the feature.

Therefore, in most software development situations it is considered good practice that when a bug is located and fixed, a test that exposes the bug is recorded and regularly retested after subsequent changes to the program. Although this may be done through manual testing procedures using programming techniques, it is often done using automated testing tools. Such a 'test suite' contains software tools that allows the testing environment to execute all the regression test cases automatically; some projects even set up automated systems to automatically rerun all regression tests at specified intervals and report any regressions.

For small projects, common tactics include running such a system after each successful build, every night, or once a week. The extreme programming software development process includes regression testing. Extensive, repeatable, and automated testing of the complete software package at every stage of the software development cycle replaces design documentation in this methodology.

Steps for regression testing

- Detect Changes in the Source Code
- Prioritize Those Changes and Product Requirements
- Determine Entry Point and Entry Criteria
- Determine Exit Point
- Schedule Tests

Types of Regression Testing

- Corrective Regression Testing
- Retest-all Regression Testing
- Selective Regression Testing
- Progressive Regression Testing

- Complete Regression Testing
- Partial Regression Testing
- Unit Regression Testing

Regression testing techniques

- Re-test All
- Regression test Selection
- Prioritization of test case

Regression Testing Tools

Selenium: This is an open source tool used for automating web applications. Selenium can be used for browser-based regression testing.

Quick Test Professional (QTP): HP Quick Test Professional is automated software designed to automate functional and regression test cases. It uses VBScript language for automation. It is a Data-driven, Keyword based tool.

Rational Functional Tester (RFT): IBM's rational functional tester is a Java tool used to automate the test cases of software applications. This is primarily used for automating regression test cases and it also integrates with Rational Test Manager.

Uses of regression testing

Regression testing can be used not only for testing the correctness of a program, but it is also often used to track the quality of its output. For instance in the design of a compiler, regression testing should track the code size, simulation time, and compilation time of the test suites.

System testing is a series of different tests and each test has a different purpose but all work to verify that all system elements have been properly integrated and perform allocated functions.

Summary

- The first level of testing is called unit testing. Unit testing focuses on verification of individual units or modules of software.
- The next level of testing is often called integration testing. Integration testing is a systematic technique for constructing the program structure while at the same time conducting tests to uncover errors associated with interfacing.
- System testing is testing conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements.
- System Testing is done after Integration Testing. This plays an important role in delivering a high-quality product.
- Integration Testing focuses on checking data communication amongst these modules. Hence it is also termed as 'I & T' (Integration and Testing), 'String Testing' and sometimes 'Thread Testing'.
- Regression Testing is nothing but a full or partial selection of already executed test cases which are re-executed to ensure existing functionalities work fine.

Keywords

Bottom-up Integration: Bottom-up integration testing begins construction and testing with atomic modules i.e., components at the lowest levels in the program structure.

Integration Testing: Integration testing is a systematic technique for constructing the program structure while at the same time conducting tests to uncover errors associated with interfacing.

Regression Testing: Regression testing is the re-execution of some subset of tests that have already been System Testing: System testing is testing conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements.

Top-down Integration: Top-down integration testing is an incremental approach to construction of program structure.

Unit Testing: Unit testing focuses on verification of individual units or modules of software. conducted to ensure that changes have not propagated unintended side effects.

Self Assessment

1. Unit testing is _____
 - A. It is a White Box testing technique
 - B. It involves the testing of each unit
 - C. It is done during the development of an application by the developers
 - D. All of above

2. What are the steps for work flow of unit testing?
 - A. Creating test cases
 - B. Reviewing test cases
 - C. Base lining test cases
 - D. All of above

3. Types of Unit testing are _____
 - A. Manual
 - B. Automated
 - C. Both manual and automated
 - D. None of above

4. White box testing and Grey box testing is part of ____
 - A. Stress testing
 - B. Regression testing
 - C. Unit testing
 - D. None of above

5. Tools used in Unit Testing are _____
 - A. PHPUnit
 - B. EMMA
 - C. JMockit
 - D. All of above

6. Incremental Testing and Stubs and Drivers are part of ____
 - A. Unit testing
 - B. White box testing
 - C. Integration testing approaches
 - D. None of above

7. What are the advantages of Top-Down Integration Testing?
 - A. Few or no drivers needed.
 - B. Fault Localization is easier.
 - C. Possibility to obtain an early prototype
 - D. All of above

8. Which is not part of Integration testing approaches _____
 - A. Top-Down Integration Testing
 - B. Black box testing
 - C. Mixed Integration Testing
 - D. Incremental Testing

9. System testing is _____
 - A. Falls under the black box testing
 - B. It is to evaluate the end-to-end system specifications
 - C. Performed to validates the complete and fully integrated software product
 - D. All of above

10. Recovery testing and Migration testing are used in____
 - A. Unit testing
 - B. Stress testing
 - C. System testing
 - D. All of above
11. Alpha and beta testing is used in____
 - A. Manual
 - B. Automated
 - C. User Acceptance testing
 - D. None of above
12. Tools used in User Acceptance testing are____
 - A. Watir
 - B. Fitness
 - C. Both watir and fitness
 - D. None of above
13. Regression testing is _____
 - A. Re-execute or re-running functional and non-functional tests
 - B. To make sure that new code changes should not have side effects on the existing functionalities
 - C. Changes in Existing functionality
 - D. All of above
14. What are the Steps for regression testing?
 - A. Detect Changes in the Source Code
 - B. Prioritize Those Changes and Product Requirements
 - C. Determine Entry Point and Entry Criteria
 - D. All of above
15. Partial Testing and Unit Testing is part of____
 - A. Stress testing.
 - B. Load testing.
 - C. Regression Testing
 - D. All of above

Answer for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. D | 2. D | 3. C | 4. C | 5. D |
| 6. C | 7. D | 8. B | 9. D | 10. C |
| 11. C | 12. C | 13. D | 14. D | 15. C |

Review Questions

1. What is unit testing? What are the advantages of unit testing?
2. Define integration testing. Explain top-down and bottom-up integration.
3. Write down the three different classes of test cases of regression test suite.
4. Discuss about system testing.
5. "Testing commences with a test plan and terminates with acceptance testing". Comment.
6. What are the advantages of regression testing?
7. Differentiate between unit testing and system testing.



Further Reading

- Books Rajib Mall, Fundamentals of Software Engineering, 2nd Edition, PHI.
- Richard Fairpy, Software Engineering Concepts, Tata McGraw Hill, 1997.
- R.S. Pressman, Software Engineering – A Practitioner’s Approach, 5th Edition, Tata
- McGraw Hill Higher education.
- Sommerville, Software Engineering, 6th Edition, Pearson Education.



Online Links

- rajeevprabhakaran.wordpress.com/2008/11/20/levels-of-testing
- <https://smartbear.com/>
- <https://www.softwaretestinghelp.com/>
- <https://www.javatpoint.com/>
- <https://www.tutorialspoint.com/>